Title: Starting off with the ARM Cortex M3
Author: Craig Duffy 27/5/14, 11/01/18, 6/1/20, 26/01/22
Module: Mobile and Embedded Devices, Secure Embedded Systems
Awards: BSc CSI, Bsc Forensic Computing, Computer Security.
Prerequisites: Some Linux skills, and basic computer architecture

This worksheet provides an introduction to the Olimex STM32-P107 board. You will find out how to connect to the board, and go through a basic introduction to its main features. You will also learn about the programs running on the lab's Linux hosts that allow you to connect to the board and program it.

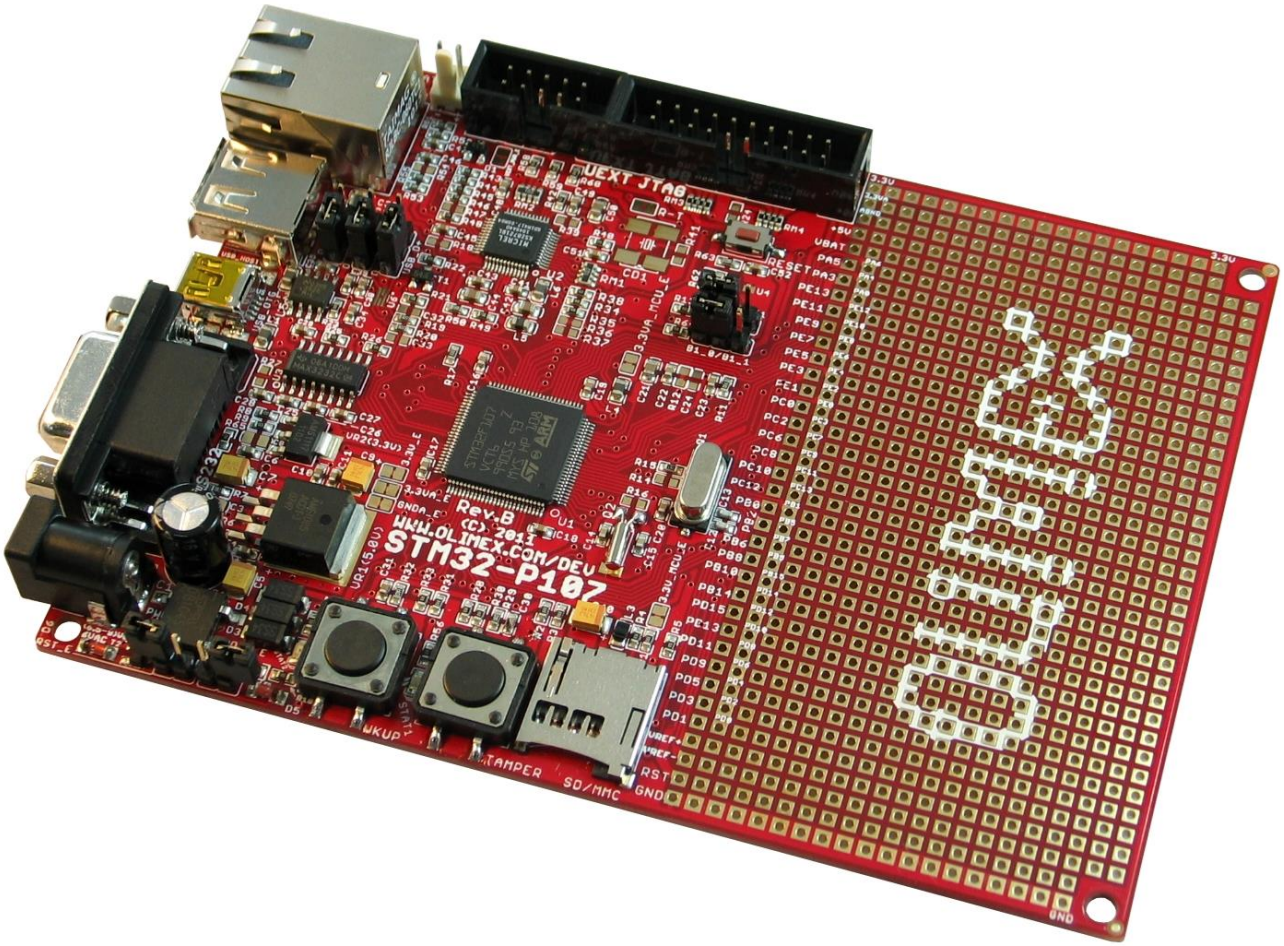
Why am I reading this?

Imagine a scenario – you have been hired by a group – think government secret service agency or Mafia styled criminal gang. They tell you to find out as much information as you can about a VIP they are interested in – maybe a senior government minister, banker or business person. You dutifully find their address, turn up to their luxury London penthouse flat, situated in an exclusive tower block. Your return to your employers and say, 'no deal boss – the building was locked, had cameras everywhere and I couldn't get in'. Well, you won't keep that job very long, in fact dependant up the organisation you might not be breathing for much longer!

What should you have done? Well find out as much as you can. Look at the plans for the building, find out what exits and entrances there are. What happens in the event of a fire? How do cleaners, porters, postal worker get in? How is the building connected to services – water, sewage and so on? If you can't get in easily, then find a similar building by the same architect and study that. These are the things that people who want information are going to do as well as people who are building things. So, we are going to be doing the same job, just this time looking at a board and its various components – namely the Olimex board.

The Olimex STM32-P107 board

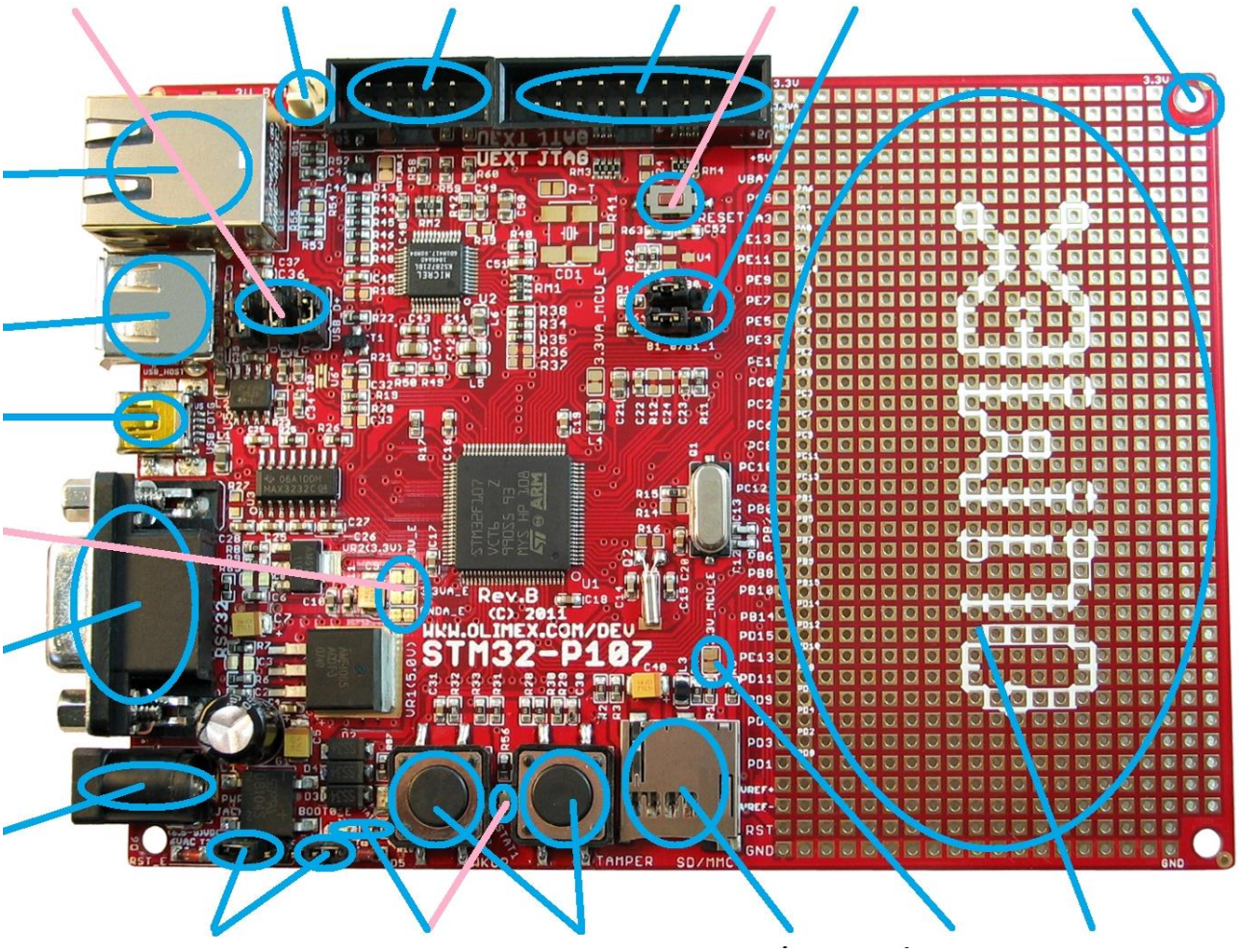
You will require a board, a USB-JTAG connector, a USB male A to B connector and a serial lead to do your work. They all should be available from the cupboard – check with your lab tutor if in doubt. If you are doing this from the project room loan equipment then all the required connectors should be in your box.



The Olimex STM32-P107 board

The STM32-P107 is a low cost board built around an ARM CORTEX M3 CPU. The CPU is fabricated by ST Microelectronics and is a STM32F107VCT6 which is one of a wide range of ARM microprocessors sold by ST.

Using the diagram below try to identify what components or features of the board that the lines are pointing to or highlighting. Note, there are some really useful pdfs in the books and documents section of learning materials on BB!



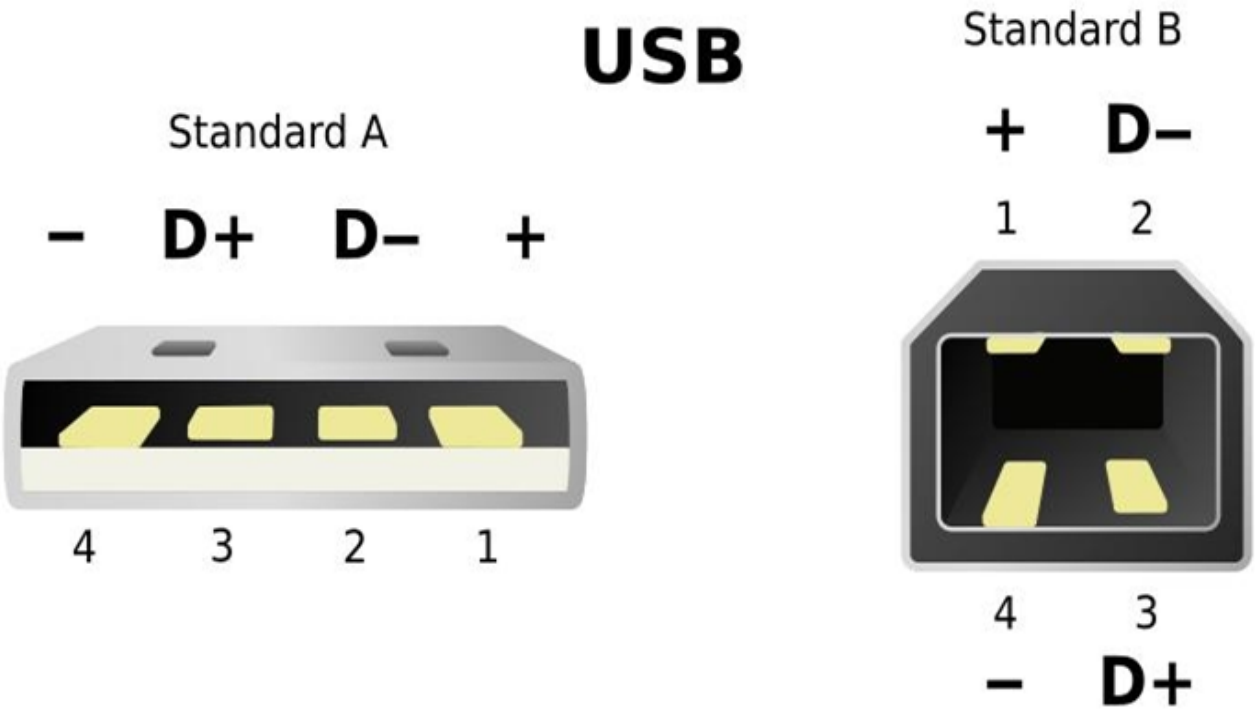
Connecting to the board

In order to do development we will require:

a) a host running the correct cross development software and
b) a cable to physically connect to the board.

Our host is the lab's Linux machines and the connection between the host and target is through a USB JTAG cable. For those working at home either a native Linux machine or virtual machine version of Linux should do – either VMWare or VirtualBox. It is possible to do this development on MacOS, but this is beyond the scope of this worksheet.

The ARM-USB JTAG connector



USB connectors

The A type connector – the flat standard USB connector – should be plugged into the Linux host, the type B connector – smaller and square – should be connected to the ARM-USB JTAG connector. The 20 way black connector should be inserted into the JTAG socket on the board – there is a notch on the side to stop you plugging it in upside down! Once the cable is connected the board will be powered, you should see a small red LED, through the USB connection. There is a separate power socket which can be used if more power is needed than USB can supply or if the board is being used without the JTAG connection. Note: If the board doesn't seem to power up ask your lab tutor as the power select jumpers (PWRSEL) affects where the power is supplied from.

Note: For this section if you are attempting this on your own system see Appendix A for details of where to get the software and how to build and install it on your system. For this worksheet you can get the required files using the git repo link on BB or from here https://gitlab.uwe.ac.uk/c-duffy/ses-worksheet-1.git. You only need to clone the link on this worksheet and there will be nothing required to push. All of the instructions for talking to the board are assuming that you have cloned the link and are executing the commands in ses-worksheet-1 directory. See appendix C for an extra note on git.

Now we have a connection we need to use some software to talk to the board. The software we will use is called OpenOCD. OCD stands for On-Chip debugger and it is a popular open source tool used for JTAG connections with a variety of boards and CPUs. It is only tangentially related to Obsessive Compulsive Disorder! This program allows the host to connect to the USB JTAG connector and any devices on the connected board. What is actually available will depend upon what support OpenOCD has for your CPU, the ARM M3 is well supported, and what the hardware designer of the board has permitted. Many commercial boards will try to hide this information as it allows users to breach security and commercially protected software.

Running OpenOCD

Start up a terminal and in the terminal change directory to the recently cloned ses-worksheet-1 directory. It is a good idea to have a Secure Embedded Systems sub-directory and to place the various cloned repos into that directory. When naming the directory don't use any spaces or special characters as these will mess up some of the script files we will use in worksheet 6. In the ses-worksheet-1 directory you will see 3 files: a README.md, FirstTest.bin and openocd.cfg. The first file is created by git, the second is a prebuilt binary to test out the board and the final file contains all the settings that OpenOCD requires to talk successfully with the board. Make sure that your terminal is running in the worksheet directory with the config file in it – you can use **pwd** (**p**rint **w**orking **d**irectory) and **ls** (**l**ist file**s**)to confirm this.

From the command line execute openocd with the following command: note the **%** character is just representing the prompt and shouldn't be typed in.

```
%openocd -f openocd.cfg
```

This should result in the following:

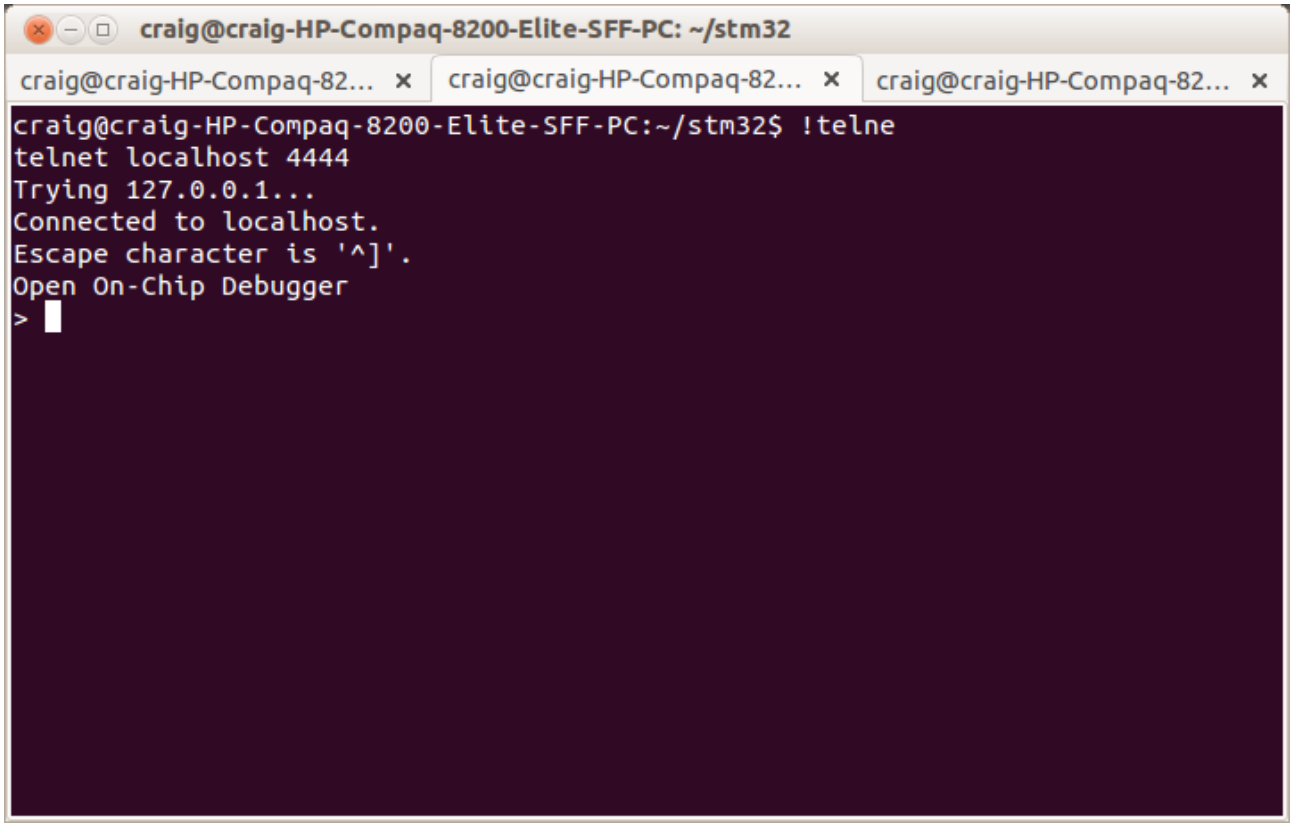
```
craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32
craig@craig-HP-Compaq-82...  craig@craig-HP-Compaq-82...  craig@craig-HP-Compaq-82...
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~$ cd stm32/
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32$ !op
openocd -f openocd.cfg
Open On-Chip Debugger 0.7.0 (2014-02-11-17:13)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.sourceforge.net/doc/doxygen/bugs.html
Info : only one transport option; autoselect 'jtag'
adapter speed: 1000 kHz
adapter_nsrst_delay: 100
jtag_ntrst_delay: 100
cortex_m3 reset_config sysresetreq
Info : max TCK change to: 30000 kHz
Info : clock speed 1000 kHz
Info : JTAG tap: stm32f1x.cpu tap/device found: 0x3ba00477 (mfg: 0x23b, part: 0
Info : JTAG tap: stm32f1x.bs tap/device found: 0x06418041 (mfg: 0x020, part: 0x
Info : stm32f1x.cpu: hardware has 6 breakpoints, 4 watchpoints
```

OpenOCD output

The output shows that openOCD has managed to talk to the CPU on the board. Once this connection has been created another program is required to talk to the openOCD process, this is normally done through either a debugger, for example **gdb**, the GNU debugger ,or through **telnet**. If you were executing and debugging code then **gdb** would be used, to simply talk to the OpenOCD process **telnet** will suffice. OpenOCD listens on port 4444 by default, so open another command window, preferably in the same directory although this isn't necessary it is normally best to run the commands all from the same directory. In the new command window execute the following command.

```
%telnet localhost 4444
```

You should get the following output:



telnet output

The **telnet** command prompt '>' means that it is waiting for command input. This means that we can issue commands to the openOCD process running on our board. At the **telnet** command prompt type the following openOCD command:

```
>target current
```

This should tell us which device we are actually talking too – and it should respond with the name **stm32f1x.cpu**. We can then find out what state the CPU is in by issuing the command.

```
>stm32f1x.cpu curstate
```

If this responds with **'running'** then it is best to type:

```
> reset halt
```

as it is easier to issue commands to a halted system. A repeat of **stm32f1x.cpu curstate**'s command should result in **'halted'** now. You can see what commands are available by type in:

```
>help
```

Which should result in the following output.

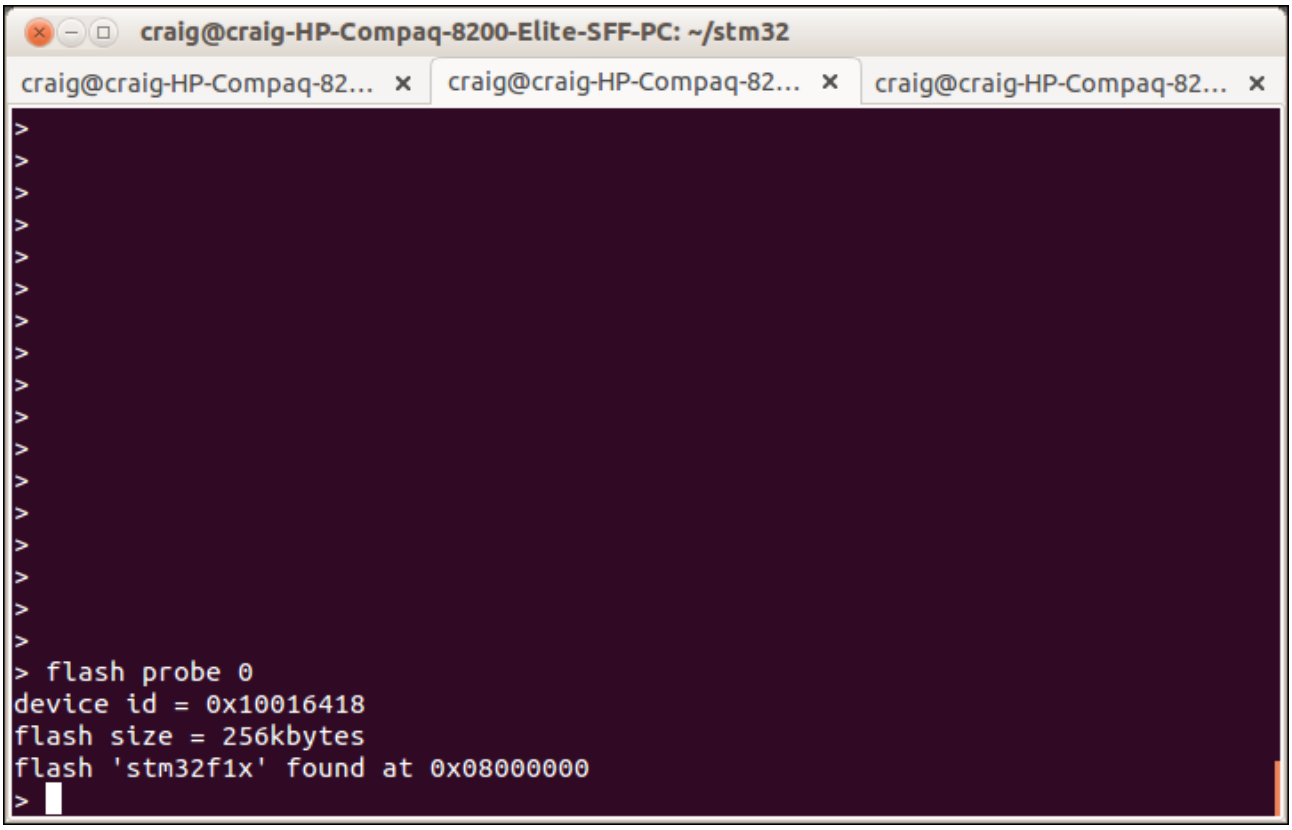
```
craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32
craig@craig-HP-Compaq-82...  craig@craig-HP-Compaq-82...  craig@craig-HP-Compaq-82...
      Display or assign flag controlling whether to verify values captured
      during IR and DR scans. (command valid any time)
version
      show program version (command valid any time)
virt2phys virtual_address
      translate a virtual address into a physical address (command valid
      any time)
wait_halt [milliseconds]
      wait up to the specified number of milliseconds (default 5) for a
      previously requested halt
wait_srst_deassert ms
      Wait for an SRST deassert. Useful for cases where you need something
      to happen within ms of an srst deassert. Timeout in ms  (command
      valid any time)
wp [address length [('r'|'w'|'a') value [mask]]]
      list (no params) or create watchpoints
xsvf (tapname|'plain') filename ['virt2'] ['quiet']
      Runs a XSVF file.  If 'virt2' is given, xruntest counts are
      interpreted as TCK cycles rather than as microseconds.  Without the
      'quiet' option, all comments, retries, and mismatches will be
      reported.
>
```

OpenOCD help command output

As you can see there are a lot of openOCD commands. Later on in the course we will start to use some of these. For the moment we can look at a common use of openOCD, to download and execute a binary built on your development host. We are now going to use the file **FirstTest.bin**, this is a pre-built simple program that flashes the LEDs on the board.

Flash commands

FirstTest.bin runs from flash memory – unlike RAM memory this memory retains its data between power cycles. In order to load the program into flash memory we need to prepare the memory first with some OpenOCD commands. First we can find out, through OpenOCD, what flash memory there is on the board, using the **flash list** command. This prints out the start address and size - strangely in decimal, 134217728 and 262144 respectively, which are 0x8000000 and 0x40000. The flash memory is arranged in several blocks (banks). We have only bank 0 so further information can be found using the **flash probe 0** command. This outputs:



Output from the flash probe 0 command

It is possible to list the state of each block of the flash memory using the **flash info 0** command. Notice from the flash info command that the blocks are not protected, generally blocks on flash devices are locked to stop accidental erasure. On our board we can erase all of the blocks with **stm32f1x mass_erase 0**. The device will only allow memory to be altered on a machine that is not running, so if the processor is not halted you will get the message - ***Target not halted stm32x mass erase failed***. What is needed is to halt the CPU with a **reset halt** command. To verify that the memory has been erased we can issue the memory word dump command **mdw 0x8000000 100**. This dumps 100 words memory from the address 0x8000000, which as we saw was the start of flash memory. This should give the following output. You will note that erasing in flash means setting the memory to logic value 1.

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm3...   craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm3...
JTAG tap: stm32f1x.cpu tap/device found: 0x3ba00477 (mfg: 0x23b, part: 0xba00, ver: 0x3)
JTAG tap: stm32f1x.bs tap/device found: 0x06418041 (mfg: 0x020, part: 0x6418, ver: 0x0)
target state: halted
target halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0xfffffffe msp: 0xfffffffc
> stm32f1x mass_erase 0
device id = 0x10016418
flash size = 256kbytes
stm32x mass erase complete
> mdw 0x8000000 100
0x08000000: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000020: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000040: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000060: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000080: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x080000a0: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x080000c0: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x080000e0: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000100: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000120: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000140: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000160: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000180: ffffffff ffffffff ffffffff ffffffff
>
```

Memory dump of formatted flash memory

Once the blocks have been erased we can load the binary image into the flash memory with the command **flash write_bank 0 FirstTest.bin 0** – this loads the binary image from the file **FirstTest.bin** into the first bank of memory with an offset of 0. Once this has been done then we can dump the memory again and see that it has changed:

```
craig@craig-HP-Compaq-8200-E...   craig@craig-HP-Compaq-8200-E...   craig@craig-HP-Compaq-8200-E...
0x080000e0: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000100: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000120: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000140: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000160: ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff
0x08000180: ffffffff ffffffff ffffffff ffffffff
> flash write_bank 0 FirstTest.bin 0
wrote 6336 bytes from file FirstTest.bin to flash bank 0 at offset 0x00000000 in 0.43405
2s (14.255 KiB/s)
> mdw 0x8000000 100
0x08000000: 20005000 08001729 080016b9 080016c5 080016cd 080016d5 080016dd 00000000
0x08000020: 00000000 00000000 00000000 080016e5 080016f1 00000000 080016fd 08001709
0x08000040: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x08000060: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x08000080: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x080000a0: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x080000c0: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x080000e0: 08001775 08001775 08001775 00000000 00000000 00000000 00000000 00000000
0x08000100: 00000000 00000000 08001775 08001775 08001775 08001775 08001775 08001775
0x08000120: 08001775 08001775 08001775 08001775 08001775 08001775 08001775 08001775
0x08000140: 08001775 08001775 08001775 08001775 00000000 00000000 00000000 00000000
0x08000160: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0x08000180: 00000000 00000000 00000000 00000000
>
```

Memory dump of file loaded into flash memory

The first part of the program is made up of various important addresses, such as the stack pointer, 0x20005000 and the vector table – addresses start 0x8000000. Now we have a program loaded we can run it – to do this type in **reset run** in the **telnet** window. This should result in the yellow and green LEDs flashing alternately – you may require a little rest after all the excitement! If it gets too much, then you can halt the program by typing **reset halt** again.

Embedded development

In order to do embedded development we need a host – often developers will work on one host with lots of different target boards and devices. The host needs to be able to communicate with the target, via USB JTAG in our case, but it also needs specialist tools and software. Some, such as openOCD, run on the host CPU, however we will also need to cross develop code that will run on the target CPU. These tools are known as tool chains and they consist of compilers, linkers and various support tools. The ones we are using are GNU tools which includes the GNU CC compiler and the binutils support tools. The tool chain we are using is the **arm-none-eabi-** one from CodeSourcery.

We can perhaps understand a little more about our cross compiler if we compare it with the host compiler. Create a very simple C program, called test.c it should be as follows:

```
main()
{
  int i;
  i = i + 1;
}
```

Now compile the program using the host gcc compiler:

%gcc -o test686.out test.c

The **-o** option puts the output into the named file rather than a.out. You will probably get some warnings about the code but you can ignore those. Then compile the same source file with the arm cross compiler:

%arm-none-eabi-gcc -o testarm.out -nostdlib test.c

We are telling the compiler not to try to use any standard libraries, -nostdlib, as it will generate some errors and you will also get a warning from the linker, **ld**, about start addresses, we can safely ignore this. Now we can use the UNIX **file** utility to tell us what if know about the 2 files. Assuming a 32-bit system we get the following:

```
%file test*
test686.out: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs),
for GNU/Linux 2.6.24, BuildID[sha1]=0x9d0fe14d3ed1784a9a6b83f629975fa484023b19, not stripped
testarm.out: ELF 32-bit LSB executable, ARM, version 1 (SYSV), statically linked, not stripped
test.c:     C source, ASCII text
```

If we use a 64-bit system we must have the relevant 32-bit libraries installed and use the -m32 switch for the gcc compilation to give an ELF 32-bit executable.

You will note that the host gcc has built a binary for an Intel CPU and the arm-none-eabi-gcc has built for ARM. You will also notice the format arm-none-eabi-gcc – this is a GNU standard for compilers and tools – the first is architecture (arm), the second operating system (none), the third calling standard (eabi or extended arm binary interface) and finally the tool (gcc). In fact if we look at the standard compiler – gcc, we can, through some investigation find out more about it. So if you type in

%whereis gcc
gcc: /usr/bin/gcc /usr/lib/gcc /usr/share/gcc /usr/share/man/man1/gcc.1.gz

If you then type – using –l for extra output.

%ls –l /usr/bin/gcc
lrwxrwxrwx 1 root root 5 Mar 20 2020 /usr/bin/gcc -> gcc-9

This is showing use that gcc is in fact a link to gcc-9 (gcc version 9). and finally, if we do

%ls –l /usr/bin/gcc-9
lrwxrwxrwx 1 root root 22 Aug 8 2020 /usr/bin/gcc-9 -> x86_64-linux-gnu-gcc-9

So, this shows us that gcc is in fact a x86_64 (64 bit pentium), generating code for Linux, using the gnu binary interface. Note the response might differ on different installations.

You should be able to execute the Intel one (it doesn't do much) the ARM one will either refuse to run or, if executed on a virtual machine will cause an error trap. If you type in on your terminal **arm-none-eabi** and press the tab character Linux will show you all the possible command invocations. On my home machine this results in:

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:/tmp$ arm-none-eabi-
arm-none-eabi-addr2line   arm-none-eabi-gcc-4.7.2   arm-none-eabi-objcopy
arm-none-eabi-ar          arm-none-eabi-gcc-ar      arm-none-eabi-objdump
arm-none-eabi-as          arm-none-eabi-gcc-nm      arm-none-eabi-ranlib
arm-none-eabi-c++         arm-none-eabi-gcc-ranlib  arm-none-eabi-readelf
arm-none-eabi-c++filt     arm-none-eabi-gcov        arm-none-eabi-size
arm-none-eabi-cpp         arm-none-eabi-gdb         arm-none-eabi-strings
arm-none-eabi-elfedit     arm-none-eabi-gprof       arm-none-eabi-strip
arm-none-eabi-g++         arm-none-eabi-ld
arm-none-eabi-gcc         arm-none-eabi-nm
```

Most of these commands have manual pages – so for arm-none-eabi-objdump type in **man objdump**. As the course progresses we will learn how to use these commands and their many functions.

Appendix A

Installing and configuring OpenOCD and the GNU tools for home use. This information is a bit out of date – ask Craig for advice if you want to do this on your own machine.

These are instructions for Ubuntu – they should work on most Debian based systems.

Install FTD libs.

sudo apt-get install libftdi-dev libftdi1 (you could try -s first to see if they are already installed)

Get OpenOCD

Get a copy of OpenOCD from http://openocd.sourceforge.net/ - I used 0.7.0 but the latest 0.8.0 should work fine.

Download and unpack. Once in the source directory type the following commands.

```
./configure --enable-maintainer-mode –enable-ft2232_libftdi
make
sudo make install
```

Install the compiler and tools

The version I used was the CodeSourcery arm-none-eabi- version 4.5.2, If you download and use other versions some parts of the makefile of linker scripts may need changing (best of luck!). The installation is fairly simple and there is a free download version, all you have to do is fill in your details and receive an email with a link for the download.

You will also need telnet which is not standard so use your application manager/tool to install that.

Appendix B openocd.cfg file

```
# openocd.cfg file for olimex boards
# using arm usb tiny jtag connector
# found by ALex edited by craig for Olimex boards
# January 2018

debug_level 2
telnet_port 4444
gdb_port 3333
source [find interface/ftdi/olimex-arm-usb-tiny-h.cfg]

#reset_config srst_only srst_open_drain
#jtag_ntrst_delay 200

#jtag_rclk 0
#jtag_rclk 8

source [find target/stm32f1x.cfg]

scan_chain
```

Appendix C git or not to git

The usage of git is important for this module and will be really useful for you in other courses and your working life. However, the worksheets will also have an option to work without git. So, for worksheet 1 that would mean downloading the FirstTest.bin file manually and creating the openocd.cfg from appendix B. In later worksheets more code will be downloaded and using git is the easiest and preferred method but there will also be tar (tape archive) files which can be used. It is worth noting that the tar files will probably go out of sync, mainly out of date, with the repo so the latter is best to use.\\